FOLHA CLARA

Entrega A1

Planejamento, Modelagem e Garantia da Qualidade

Identificação da entrega

Projeto: Folha Clara - Sistema de Folha de Pagamento

Unidade curricular: Garantia da Qualidade de Software

Avaliação: A1 - Planejamento, Modelagem e Garantia da Qualidade

Prazo oficial: 20 de outubro de 2026

Versão dos artefatos: 1.0 - 29 de setembro de 2026

Professor: Paulo Cesar Furlanetto Marques

Integrante	RA
Felipe Vergara Chico	1292416373
Nicolas Uliana Ramos	1292413395
Victor Pithan Altnetter	1292414778
Miguel Marques Durte	1292413635

Nomes e RAs informados pela equipe. A distribuição nominal proposta consta do planejamento; ela organiza o trabalho futuro e não atribui autoria retroativa.

Fontes: Trabalho_Integrador_A1_A3_GQS.pdf e T1 - Requisitos do Cliente.docx. Baseline técnica: código e evidências disponíveis neste pacote.

A1 - Índice e checklist de entrega

Fonte: **Trabalho_Integrador_A1_A3_GQS.pdf**, seção 5 (páginas 2 e 3), estrutura da seção 7 e critérios da seção 8. Requisitos de produto: **T1 - Requisitos do Cliente.docx**. Documentos produzidos para a A1 em 29/09/2026.

Item	Entregável	Arquivo editável	Critério de conferência
5.1	Visão	01_Visao.md	Objetivo, escopo, stakeholders, premissas, restrições e benefícios
5.2	ERS	02_ERS.md e 03_Rastreabilidade.md	9 RFs, 8 RNs, 4 RNFs, aceitação e vínculos
5.3	UML	04_Modelagem.md e diagramas/	Casos de uso, classes e sequência; desenhos no PDF
5.4	Arquitetura	05_Arquitetura.md	Tecnologias, componentes, decisões e persistência
5.5	PGQ	06_PGQ.md	Padrões, versões, revisão, métricas e aprovação
5.6	Plano de testes	07_Plano_Testes.md	Estratégia, níveis, ambiente, entrada e saída
5.7	Casos de teste	08_Casos_Teste.md	30 casos no template CT, com requisito e evidência
5.8	Planejamento	09_Planejamento.md	EAP, cronograma, dependências e responsabilidades
5.9	Repositório	README.md da raiz e evidencias/historico_commits.txt	Execução, estrutura inicial e histórico real

O PDF consolidado é a versão de leitura. Os Markdown, JSON e PlantUML são editáveis e acompanham o ZIP. O código é incluído sem dependências instaladas, credenciais, dados locais ou builds. Os diretórios da A3 recebem indicação de fase futura, sem afirmar que já houve homologação.

Conferência antes de enviar

1. Conferir a identificação da equipe e a distribuição proposta dos papéis.
2. Revisar as decisões e critérios operacionais da ERS com o professor/cliente.
3. Confirmar o acesso do avaliador ao repositório Git original.
4. Se houver alterações, executar as verificações e regenerar o pacote.
5. Enviar **output/Entrega_A1_Folha_Clara.zip** até 20/10/2026.

O registro dos testes atuais é evidência complementar. A A1 exige especificação de pelo menos 20 casos; os testes de usabilidade e de extensibilidade aqui planejados ainda não representam aceite de usuário ou homologação da A3.

1. Documento de visão do sistema

Objetivo

Centralizar o cadastro de colaboradores e calcular automaticamente a remuneração das três categorias do cliente, permitindo consultar dados, emitir folhas individuais e consolidadas e conhecer o custo total da folha.

Escopo incluído

Cadastro com matrícula, nome, tipo e salário base; vendas e percentual para Comissionado; quantidade e valor unitário para Produção; consulta, alteração e exclusão; validação; cálculo automático; persistência no backend; folha com base, adicional e salário final; resumo com quantidade, total geral e total por categoria. RF009, RN001 e resumo por categoria, recomendados pelo cliente, foram adotados no escopo da equipe.

Fora do escopo

Tributos, INSS, FGTS, IRRF, férias, 13º salário, descontos, integração bancária, controle de ponto, autenticação, múltiplas empresas e histórico mensal. Não se interpreta a remuneração acadêmica como uma folha legal ou salário líquido. Inclusões futuras precisam de avaliação de impacto em ERS, arquitetura e testes.

Partes interessadas

Parte	Interesse	Participação esperada
Administrador / operador de RH	Cadastrar e obter valores corretos	Validar fluxos e clareza das mensagens
Gestor	Conhecer custo total e por categoria	Conferir resumo e relatórios
Colaborador	Dados e remuneração corretos	Interessado nos resultados; sem acesso próprio previsto
Professor / cliente acadêmico	Conformidade e evidências	Avaliar artefatos e esclarecer ambiguidades
Equipe de desenvolvimento	Solução testável e sustentável	Implementar, revisar, testar e documentar

Premissas

Operador fornece dados corretos e possui acesso autorizado à máquina; uma API local gerencia um arquivo JSON; Node e navegador estão disponíveis; dados de demonstração são fictícios; valores monetários são apresentados em reais. A percentagem informada como 5 significa 5%, convenção explicitada na ERS.

Restrições

A1 até 20/10/2026; A3 até 01/12/2026. A entrega A1 prioriza planejamento, modelagem e qualidade. Persistência em JSON é a decisão atual do projeto. Não há suporte garantido a escrita concorrente entre processos nem controle de acesso por usuário. A documentação deve refletir a implementação inspecionada, sem atribuir aprovação a testes manuais ainda não realizados.

Benefícios e indicadores

Reduzir cálculo manual (três fórmulas automatizadas); reduzir inconsistências (validação das oito regras de negócio); facilitar conferência (folha individual, consolidada e totais); permitir auditoria acadêmica (cada requisito ligado a critério de aceitação, caso de uso, componente e caso de teste). O benefício é esperado; não foi medido ganho de produtividade com usuários reais.

2. Especificação de requisitos de software

Origem e convenções

Baseline do produto: **T1 - Requisitos do Cliente.docx**, seções 1 a 9. Os IDs RF001-RF009, RN001-RN008 e RNF001-RNF004 foram preservados. Critérios CA e decisões D abaixo operacionalizam a especificação para teste; não são novos requisitos atribuídos ao cliente. Prioridade alta para todos os RFs; RF009 e RN001 são recomendações do cliente adotadas pela equipe.

Administrador é o ator que opera o sistema; não implica login implementado. Salário base é a parcela fixa; adicional é comissão ou produtividade; salário final é base mais adicional. Matrícula é identificador de negócio; UUID **id** é a identidade técnica. Valores de entrada são números não negativos.

Requisitos funcionais e critérios de aceitação

ID	Especificação	Critério de aceitação verificável
RF001	Cadastrar matrícula, nome, salário base e tipo	CA001: dados válidos geram HTTP 201, identidade e cadastro consultável; entradas inválidas são rejeitadas
RF002	Cadastrar colaborador Padrão	CA002: base 1800 resulta em adicional 0 e final 1800
RF003	Cadastrar Comissionado com vendas e percentual	CA003: vendas 10000 e percentual 5 são armazenados; com base 2000, final 2500
RF004	Cadastrar Produção com quantidade e valor por unidade	CA004: quantidade 80 e valor 12,50 são armazenados; com base 1500, final 2500
RF005	Listar dados cadastrais e remuneratórios	CA005: listar e consultar retornam matrícula, nome, tipo, base, adicional e final calculado
RF006	Calcular salário automaticamente conforme categoria	CA006: base 1000 retorna 1000 para Padrão, 1200 com vendas 2000 a 10%, 1100 com 20 unidades a 5
RF007	Gerar folha para todos os colaboradores	CA007: folha consolidada contém cada cadastro e seus valores; detalhe individual retorna apenas o colaborador solicitado
RF008	Apresentar total geral da folha	CA008: salários 1800 e 1500 resultam em total 3300 e quantidade 2; totais por categoria reconciliam com o geral
RF009	Incluir, alterar, excluir e consultar	CA009: alteração persiste e recalcula preservando identidade; exclusão remove consulta e participação na folha

Folha individual e detalhamento de base/adicionais derivam das seções 2, 6 e 9 do cliente e são vinculados a RF005-RF007. Quantidade e totais por categoria derivam da seção 6 e complementam RF008. Não se criaram RFs fictícios para eles.

Regras de negócio

ID	Regra do cliente	Critério de aceitação
RN001	Matrícula única	CA-RN001: duplicata no cadastro ou alteração retorna 409; dados anteriores são mantidos
RN002	Nome obrigatório	CA-RN002: nome vazio ou somente espaços retorna 400
RN003	Salário base não negativo	CA-RN003: -0,01 rejeitado; zero permitido
RN004	Vendas não negativas	CA-RN004: vendas -0,01 rejeitadas para Comissionado
RN005	Comissão percentual não negativa	CA-RN005: percentual -0,01 rejeitado para Comissionado
RN006	Quantidade produzida não negativa	CA-RN006: quantidade -1 rejeitada para Produção
RN007	Valor por unidade não negativo	CA-RN007: valor -0,01 rejeitado para Produção
RN008	Salário final automático	CA-RN008: calcular e recalcular pelo domínio; salário final enviado pelo cliente não determina a remuneração

Fórmulas e decisões de interpretação

- D01: Padrão: final = base. Comissionado: adicional = vendas × percentual / 100; final = base + adicional. Produção: adicional = quantidade × valor unitário; final = base + adicional.

- D02: API recebe percentual em pontos percentuais (5 significa 5%). A fórmula do cliente usa a taxa; dividir por 100 converte o dado de entrada. Não se impõe limite superior de 100%, pois não foi especificado.

- D03: base e adicionais são apresentados com duas casas. O código arredonda adicional e final com **Math.round((valor + Number.EPSILON) * 100) / 100**. Não se declara precisão financeira arbitrária.

- D04: matrícula é aparada e comparada sem distinguir maiúsculas; acentos continuam significativos (**sensitivity: accent**). Alteração exige cadastro completo.

- D05: folha representa o estado atual, sem competência ou histórico mensal. Exclusão elimina o registro da folha atual.

- D06: faltam limites de magnitude e escala numérica no documento do cliente. Revisar com o cliente antes de alegar suporte a valores extremos ou regras trabalhistas.

Requisitos não funcionais

ID	Texto / intenção do cliente	Critério operacional proposto pela equipe
RNF001	Interface amigável e intuitiva	CA-NF001: operador percorre cadastro, consulta e folha; campos têm rótulos, erro é compreensível, dados são legíveis em 1366×768 e 390×844; CT-029 manual
RNF002	Validar dados antes de calcular	CA-NF002: entradas obrigatórias, tipos e números inválidos retornam 400 antes de gravar; arquivo permanece idêntico
RNF003	Armazenamento persistente	CA-NF003: nova instância da API usando o mesmo JSON recupera os cadastros e cálculos; CT-024
RNF004	Novos tipos sem mudar cálculos existentes	CA-NF004: nova subclasse pode implementar adicional e ser registrada na fábrica/tipos/validação sem editar fórmulas das três classes existentes; CT-030 de revisão

RNF001 não é comprovado por cobertura do backend. RNF004 permite alterar pontos de registro e formulários; não promete extensão sem qualquer modificação. Critérios operacionais e decisões D01-D06 devem ser confirmados na revisão da baseline. Critérios de aprovação documental estão no PGQ; aceite final é da A3.

3. Matriz de rastreabilidade

Cada um dos 21 requisitos do cliente possui critério na ERS e caso de teste. Ter um vínculo não equivale a homologação. CT-029 e CT-030 estão planejados; os demais têm evidência automatizada, com agrupamentos explicados nos CTs.

Requisito	Aceitação	Caso de uso	Componente backend / frontend	Casos de teste
RF001	CA001	UC01	colaboradorController, validações / EmployeesPage	CT-001, CT-010, CT-027
RF002	CA002	UC01, UC07	ColaboradorPadrao / CompensationFields	CT-002, CT-020
RF003	CA003	UC01, UC07	ColaboradorComissionado / CompensationFields	CT-003, CT-006
RF004	CA004	UC01, UC07	ColaboradorProducao / CompensationFields	CT-004, CT-006
RF005	CA005	UC02, UC08	listar/buscarColaborador / EmployeesList	CT-005, CT-017, CT-024
RF006	CA006	UC07	models + ColaboradorFactory	CT-006, CT-021, CT-028
RF007	CA007	UC03, UC08	folhaPagamentoService / PayrollPage, IndividualPayrollPage	CT-007, CT-017, CT-019
RF008	CA008	UC04	gerarFolhaPagamentoService / PayrollSummary	CT-008, CT-018, CT-019
RF009	CA009	UC01, UC02, UC05, UC06	colaboradorService / EmployeeMaintenancePanel	CT-009, CT-021, CT-022, CT-023
RN001	CA-RN001	UC01, UC05	colaboradorService	CT-010, CT-022
RN002	CA-RN002	UC01, UC05	colaboradorValidations	CT-011
RN003	CA-RN003	UC01, UC05	colaboradorValidations	CT-012, CT-020
RN004	CA-RN004	UC01, UC05	colaboradorValidations	CT-013, CT-025
RN005	CA-RN005	UC01, UC05	colaboradorValidations	CT-014
RN006	CA-RN006	UC01, UC05	colaboradorValidations	CT-015
RN007	CA-RN007	UC01, UC05	colaboradorValidations	CT-016
RN008	CA-RN008	UC07	Colaborador e subclasses	CT-002, CT-006, CT-027, CT-028
RNF001	CA-NF001	UC01-UC08	pages, components, styles	CT-029 (manual planejado)
RNF002	CA-NF002	UC01, UC05	middlewares + validations	CT-011 a CT-016, CT-025, CT-027
RNF003	CA-NF003	UC01, UC02	ColaboradorRepository	CT-020, CT-024, CT-026
RNF004	CA-NF004	UC07	subclasses + fábrica + tipos	CT-030 (revisão planejada)

Referências de código: **backend/src/** e **frontend/src/**. A especificação dos CTs inclui o caminho e o título exato do teste automatizado, permitindo o caminho inverso teste → CT → requisito. 21/21 requisitos estão rastreados. 19/21 possuem cenários automatizados associados; RNF001/RNF004 ainda dependem das verificações planejadas. Esse indicador não significa 19 requisitos formalmente homologados ou cobertura integral de cada requisito.

4. Modelagem UML

Três diagramas aparecem desenhados no PDF e possuem fontes PlantUML em **diagramas/**. A modelagem representa o código inspecionado em 29/09/2026.

Casos de uso

UC01-UC04 preservam os nomes do cliente. UC05-UC08 detalham manutenção, cálculo e folha individual. Um único ator, Administrador, representa o operador.

Caso	Requisitos	Fluxo principal e pós-condição	Alternativas
UC01 Cadastrar	RF001-RF004, RN001-RN008	Informar identificação; selecionar tipo; preencher remuneração; confirmar; validar, calcular e persistir	Entrada inválida: 400; duplicata: 409; nenhum cadastro novo
UC02 Consultar	RF005	Abrir colaboradores; solicitar lista/detalhe; recuperar e calcular dados; exibir	Lista vazia: 200 com []; matrícula ausente: 404
UC03 Gerar folha	RF006, RF007	Solicitar folha; ler registros; executar UC07 para cada item; exibir detalhamento	Nenhum cadastro: itens vazios; falha de persistência: erro
UC04 Emitir resumo	RF008	Solicitar resumo; calcular folha; somar quantidades, valores e categorias; exibir	Base vazia: totais e quantidades zero
UC05 Alterar cadastro	RF009	Consultar matrícula; editar dados completos; validar; recalcular; persistir mantendo id	Ausente: 404; matrícula em uso: 409; inválido: 400
UC06 Excluir cadastro	RF009	Selecionar registro; confirmar na interface; excluir; atualizar consulta/folha	Cancelamento não envia exclusão; ausente: 404
UC07 Calcular remuneração	RF006, RN008	Instanciar categoria; calcular adicional; arredondar; obter final	Categoria inválida é bloqueada pela validação
UC08 Emitir folha individual	RF005-RF007	Selecionar matrícula; buscar e calcular colaborador; exibir folha individual	Ausente: 404; matrícula vazia: 400

Pré-condições comuns: aplicação disponível, JSON acessível e entradas válidas para o fluxo de sucesso. UC05/UC06/UC08 exigem cadastro existente. UC07 é incluído pelos fluxos que calculam; não é uma ação separada na interface.

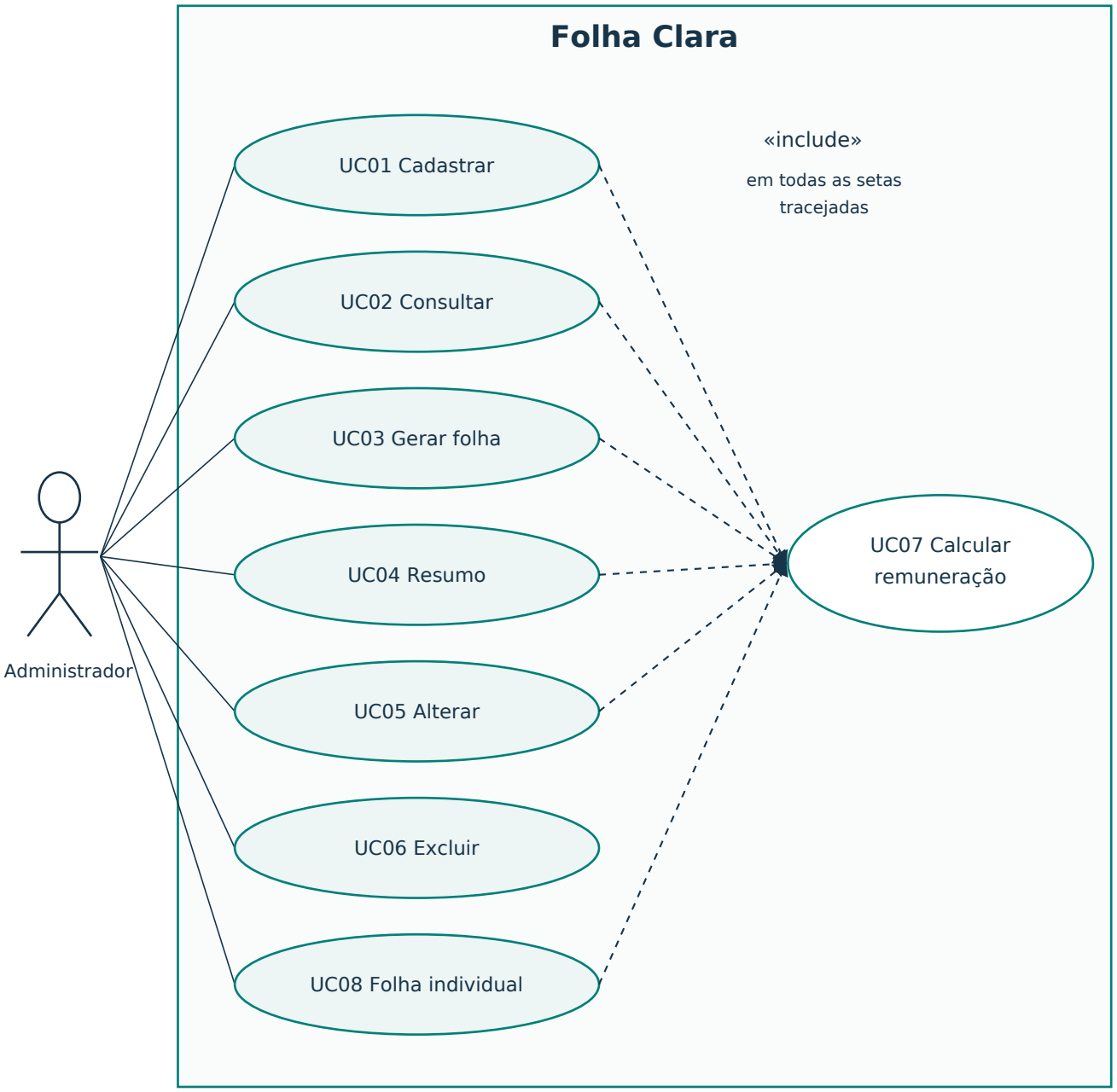
Classes

Colaborador contém identificação, base, adicional e final, além de **calcularAdicional()** e **definirRemuneracao()**. É classe concreta no código. **ColaboradorPadrao**, **ColaboradorComissionado** e **ColaboradorProducao** herdam dela. A fábrica cria a classe adequada. O repositório persiste registros JSON. Services e controllers são módulos com funções, não classes; foram omitidos do diagrama de classes e aparecem na sequência e arquitetura.

Sequência

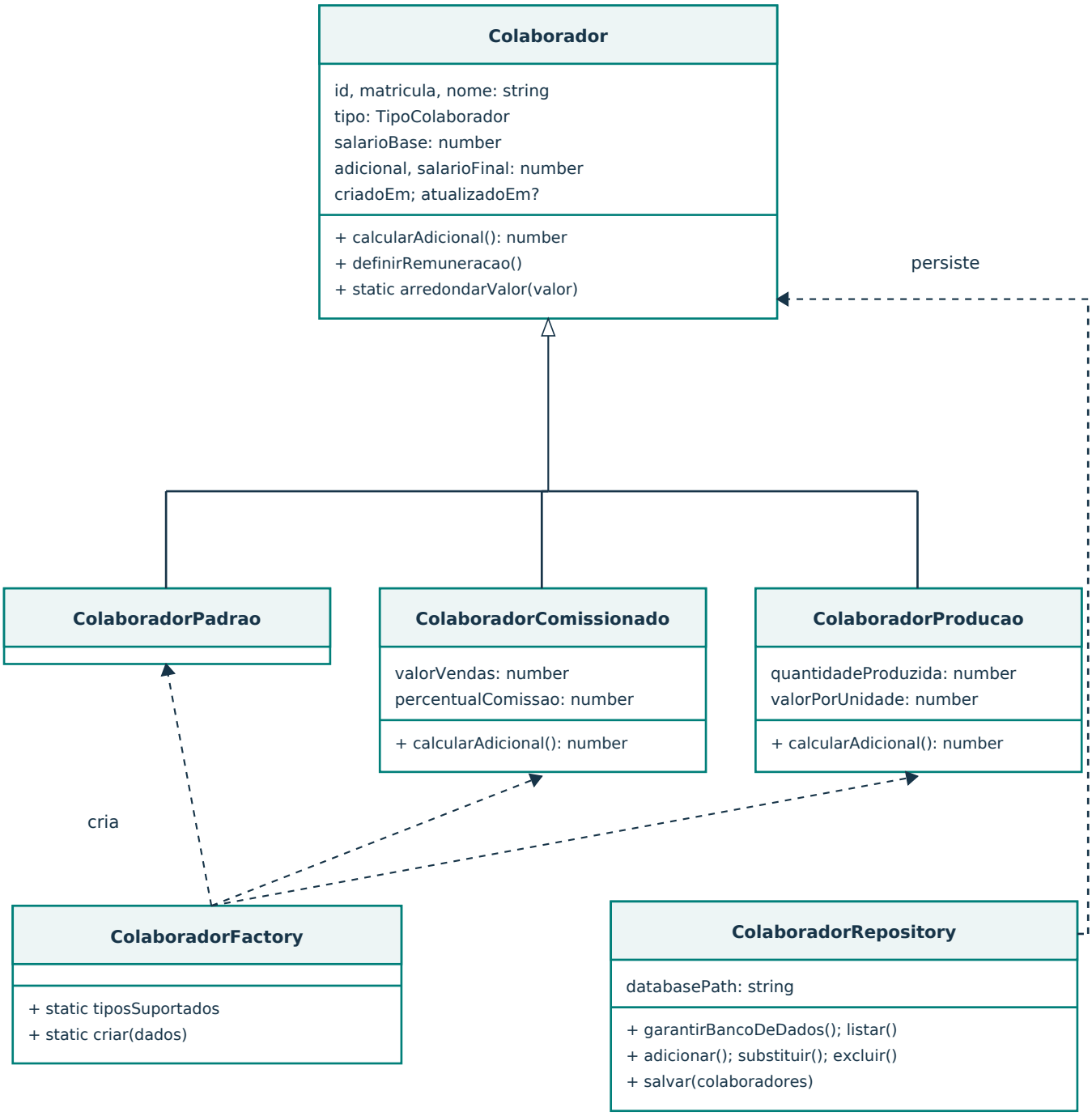
Fluxo escolhido: cadastro comissionado (UC01), pois atravessa interface, validação, serviço, fábrica, modelo e persistência. O fragmento alternativo distingue sucesso (201), entrada inválida (400) e matrícula duplicada (409). A validação ocorre antes da criação do modelo e da gravação.

4.1 Diagrama de casos de uso



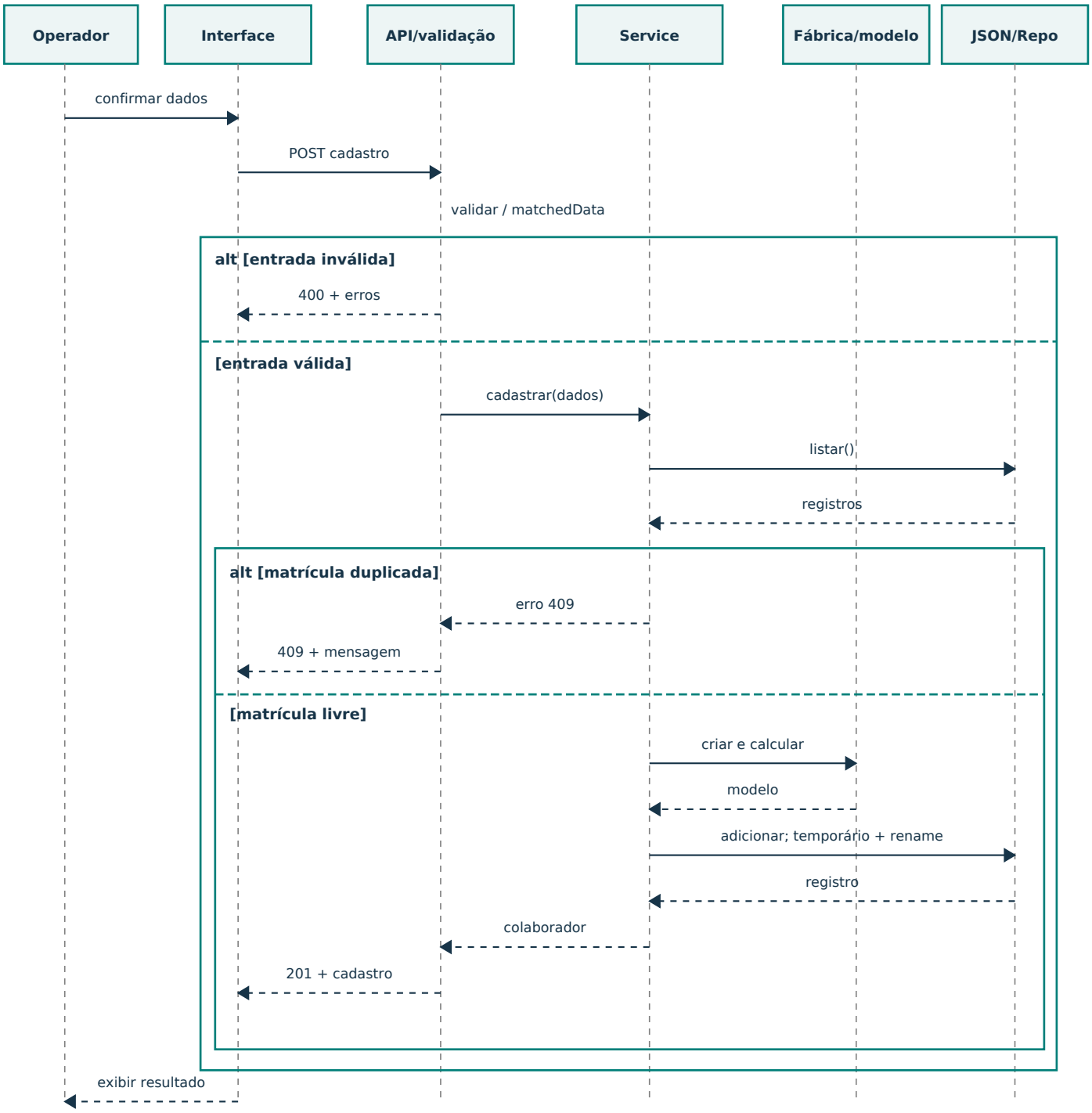
Linhas contínuas: associação com ator. Setas tracejadas: dependência «include». Exclusão não inclui cálculo no próprio endpoint.

4.2 Diagrama de classes



Triângulo vazio: generalização. Setas tracejadas: dependência. Atributos de remuneração especializados por categoria. Métodos são resumidos; fonte PlantUML acompanha o pacote.

4.3 Diagrama de sequência - UC01



Retornos tracejados; mensagens contínuas. Fragmentos alt distinguem erro de entrada, duplicidade e sucesso. API agrupa rota, validação, controller e tratamento de erro; JSON/Repo agrupa persistência.

5. Documento de arquitetura

Estilo e tecnologias

Aplicação cliente-servidor com API REST e backend em camadas. Frontend React 19, React Router 7, TypeScript 6 e Vite 8. Backend Node.js, Express 5, TypeScript 6, express-validator 7 e dotenv 17. Testes com Vitest 5 e cobertura V8; análise estática com ESLint 10, TypeScript estrito e SonarJS. Versões exatas resolvidas estão nos **package-lock.json**; Node mínimo documentado: 22.13.

Componentes e dependências

Camada	Caminho	Responsabilidade
Interface	frontend/src/pages e components	Formulários, listagem, navegação, estados e impressão
Orquestração cliente	frontend/src/hooks	Carregar, atualizar e apresentar sucesso/erro
Comunicação	frontend/src/services	Fetch, contratos HTTP e base da API
Entrada HTTP	backend/src/routes e controllers	Rotas, parâmetros e respostas
Validação	backend/src/validations e middlewares	Normalizar tipo, conferir entradas e extrair dados permitidos
Negócio	backend/src/services	Unicidade, manutenção, consultas e composição da folha
Domínio	backend/src/models	Classes de remuneração e fábrica
Persistência	backend/src/data	Contexto por aplicação, parser e repositório JSON
Erros	backend/src/errors	Erros de negócio e respostas centralizadas

Fluxo: página → hook → serviço HTTP → rota → validação → controller → serviço de negócio → fábrica/modelos + repositório. O servidor é composto em **app.ts**. **createApp** recebe caminho de dados e não abre porta; **startServer** inicia HTTP. O repositório pertence à aplicação (**app.locals**), permitindo testes isolados.

Contratos principais

/api/colaboradores: POST cadastra (201), GET lista (200); **/:matricula**: GET consulta (200), PUT substitui dados (200), DELETE exclui (204). Bases por categoria: **colaboradores-padrao**, **colaboradores-comissionados** e **colaboradores-producao**, com o mesmo CRUD e isolamento de tipo. **GET /api/folha-pagamento** retorna **{itens, resumo}**; **GET /api/folha-pagamento/:matricula** retorna um item detalhado. Erros: 400 entrada inválida, 404 ausente, 409 matrícula duplicada, 500 falha interna. API usa **mensagem** e, nas validações, **errors** com campo/mensagem.

Modelo de persistência

Arquivo padrão: **backend/data/colaboradores.json**, lista de objetos contendo **id**, **matricula**, **nome**, **salarioBase**, **tipo**, **criadoEm**, opcional **atualizadoEm**, adicional, salário final e campos específicos da categoria. **COLABORADORES_DB_PATH** pode substituir o caminho. Arquivo inexistente é criado como **[]**. Leitura valida formato e registros; conteúdo inválido gera erro. Consultas recriam o modelo e recalculam remuneração, não confiam nos totais armazenados. Escrita usa arquivo temporário no mesmo diretório e rename.

Não há scripts SQL porque não há SGBD. O pacote registra essa decisão em **06_Banco_Dados/README.md**. Não se distribuem os registros locais da equipe. Backup proposto: parar a API, copiar o JSON válido, armazenar cópia com data; restauração com API parada e validação pela consulta. Procedimento a ensaiar na A3.

Decisões e justificativas

Decisão	Razão	Consequência / limite
React + API separada	Separar apresentação de regras	Contratos devem permanecer compatíveis
TypeScript estrito	Detectar incompatibilidades antecipadamente	Validação de dados externos continua necessária
Herança e fábrica	Compartilhar identificação e especializar fórmulas	Novos tipos exigem atualizar pontos de registro
JSON local	Baixo custo de instalação e escopo acadêmico	Sem transações entre processos ou consultas de grande volume
Escrita temporária + rename	Evitar exposição de conteúdo parcialmente escrito	Não substitui backup nem resolve múltiplos escritores
HTTP + diretórios temporários nos testes	Exercitar contrato e persistência reais	Ambiente de execução precisa permitir portas locais

Implantação e limites

Desenvolvimento: navegador → Vite (5173) → proxy **/api** → Express (3000) → JSON. Build frontend gera arquivos estáticos; backend gera **dist/app.js** para Node. Uma publicação exigiria hospedagem estática/proxy e configuração de origem. Sem autenticação ou autorização implementadas, a versão é de demonstração local. O modelo atual não oferece rastreio histórico de folha ou concorrência robusta.

6. Plano de Garantia da Qualidade (PGQ)

Objetivo e abrangência

Demonstrar conformidade com a ERS e prevenir regressões nos cálculos, validações, relatórios e persistência. Abrange documentos A1, frontend, backend e testes. O plano define práticas adotadas para novas alterações; não presume que todas as revisões históricas já seguiram este processo.

Padrões de codificação

TypeScript estrito; ES Modules; componentes React em TSX; funções e arquivos coesos; nomenclatura de domínio em português; imports locais backend com **.js** para execução compilada. Controllers tratam HTTP, services orquestram negócio, models calculam e repositório grava. Não duplicar cálculos no frontend. Executar ESLint/SonarJS e typecheck antes de revisar. Não incluir credenciais, dados reais, dependências instaladas ou relatórios temporários nos commits.

Controle de versões e mudança

Git é a fonte de histórico. Usar branch curta por requisito/correção, commits descritivos e pull request com objetivo, requisito, impacto e evidências. Antes de integrar: revisor verifica diff, testes, ERS e rastreabilidade. Baseline A1: documentação 1.0; qualquer alteração de requisito registra motivo, autor, impacto, decisão e atualização dos CTs. Tags/releases são decisões da equipe; este pacote não cria retrospectivamente commits, revisões ou aprovações.

Processo de revisão técnica

1. Autor prepara alteração e executa verificações aplicáveis.

2. Revisor confere requisito, entradas negativas/limites, cálculo, persistência, tratamento de erro e clareza da interface.

3. QA confere critérios de aceitação, rastreabilidade e evidência reproduzível.

4. Não conformidades recebem ID, descrição, severidade, prioridade, responsável e condição de reteste.

5. Autor corrige; revisor/QA retesta e fecha com evidência; coordenador aceita a baseline documental.

Papéis podem acumular pessoas em equipe pequena; quem revisa deve ser distinto do autor quando possível. Aprovação nominal e revisões ainda não realizadas permanecem pendentes, não são simuladas neste documento.

Métricas e critérios

Métrica	Cálculo	Meta / frequência
Rastreabilidade	Requisitos com CA e CT / 21 requisitos × 100	100% na baseline A1
Casos especificados	Contagem de IDs CT únicos	Pelo menos 20; pacote contém 30
Aprovação automatizada	Testes aprovados / executados × 100	100% antes de integrar; ignorados reportados à parte
Cobertura backend	Linhas, instruções, funções e ramos cobertos / totais	>=80% por arquivo em cada métrica
Qualidade estática	Erros em typecheck e lint	Zero; verificar backend e frontend
Build	Processos concluídos com código zero	Ambos os projetos compilam
Defeitos abertos	Contagem por severidade	Zero crítico/alto para aceite; demais com decisão explícita
Documentação	Itens 5.1 a 5.9 conferidos / 9	100% de conteúdo; identificação preenchida antes do envio

Cobertura mede execução de código, não comprova todos os comportamentos nem a usabilidade. O frontend não possui cobertura automatizada declarada. Defeitos de ambiente devem ser separados de falhas reproduzíveis do produto.

Gestão de defeitos e risco

Modelo de registro: DEF-XXX; requisito/CT; ambiente; passos; esperado; obtido; severidade; prioridade; responsável; status; correção/commit; evidência de reteste. Estados: Aberto → Em análise → Em correção → Em reteste → Fechado (ou Reaberto). Crítico: perda/corrupção de dados; alto: cálculo ou fluxo principal incorreto; médio: fluxo secundário; baixo: apresentação sem impedir tarefa. Prazo proposto: críticos/altos resolvidos antes da próxima baseline; demais triados na revisão semanal. O registro começa sem defeitos de produto formalmente triados; isso não é uma declaração de ausência de defeitos.

Aprovação da A1

Exigir visão, ERS, UML, arquitetura, PGQ, plano de testes, >=20 CTs, planejamento e repositório verificáveis. Conferir os 21 requisitos na matriz, coerência das fórmulas, autoria e responsabilidades. Testes manuais planejados podem estar não executados na A1, desde que isso esteja explícito. Homologação e critérios finais da A3 serão avaliados em fase própria.

7. Plano de testes

Escopo

RF001-RF009, RN001-RN008 e RNF001-RNF004: cadastro das três categorias, manutenção, cálculos, folhas, totais, entradas inválidas, persistência, usabilidade e extensibilidade. Não se planejam testes de tributos, login, folha mensal ou integrações não especificadas.

Estratégia e níveis

Testar caminhos positivos, classes de equivalência (válido, vazio, inválido), fronteiras (zero e negativo), duplicidade e ciclo de vida do cadastro. Verificar valores esperados por cálculo independente e dados fictícios. Executar a suíte completa após mudanças que afetem contratos, modelos ou dados.

Nível / técnica	Situação	Ferramenta / evidência
Unidade	Parser, contexto e repositório testados diretamente	Vitest, cobertura V8
Integração	API real + validação + domínio + JSON	Vitest e fetch, portas e diretórios temporários
Funcional por requisito	Um teste nominal por RF001-RF009	requisitosFuncionais.test.ts
Aceitação da interface	Planejada; não executada nesta entrega	CT-029, observação e registro pelo operador
Revisão de extensibilidade	Planejada; critérios explicitados	CT-030, revisão de classes e pontos de registro

Os testes RF são de integração funcional; não são todos unitários. Casos de relatório individual e resumo por categoria estão na suíte de relatórios. Automação atual não comprova impressão visual, acessibilidade ou tarefas de usuário no navegador.

Ambiente e ferramentas

Node >=22.13, npm, TypeScript, Vitest, V8, ESLint e navegador moderno. Execução local em Linux; frontend Vite e API Express. Dados em diretórios temporários separados do JSON de desenvolvimento, removidos ao encerrar. HTTP local usa porta livre; não precisa de internet para executar testes com dependências já instaladas. Instalação requer acesso ao registry configurado.

Procedimento reproduzível

1. Na raiz, instalar dependências com **npm ci** em backend e frontend conforme README.

2. No backend, executar **npm run test:coverage**; examinar testes e limites por arquivo.

3. Executar **npm run typecheck**, **npm run lint** e **npm run build** em cada projeto.

4. Para evidência legível por máquina: adicionar **-- --reporter=default --reporter=json --outputFile=/caminho/resultados.json** ao comando de cobertura.

5. Registrar data, revisão Git, comando, resultado, falhas e cobertura; relacionar ao CT.

6. Para CTs manuais, iniciar frontend/API com dados fictícios e registrar o observado sem preencher antecipadamente o resultado.

CT001-009 são independentes. Os CTs adicionais documentam cenários das suítes existentes: alguns compartilham preparação sequencial dentro do arquivo. Ao reproduzir evidências automatizadas, executar a suíte completa. Para reprodução manual/API isolada, preparar exatamente a pré-condição indicada em cada CT.

Entrada, saída e aprovação

Entrada: ERS disponível, dependências instaladas, código compilável, JSON de teste isolado e permissão para HTTP local. Bloqueios do ambiente são registrados e resolvidos antes de classificar falha como defeito do produto. Saída da rodada automatizada: todos os testes executados, zero falhas e zero ignorados, cobertura >=80% por arquivo, verificações estáticas e builds aprovados. Saída A1: pelo menos 20 CTs completos e rastreáveis, resultados disponíveis registrados com evidência, testes futuros identificados como não executados. Aceite A3: executar também os testes manuais planejados e obter aceite do cliente.

Suspensão, retomada e registro

Suspender se o ambiente não abre porta ou o dado de preparação está incorreto. Preservar o diagnóstico; corrigir o ambiente e repetir a rodada pertinente. Falha de asserção reproduzível gera registro de defeito pelo modelo do PGQ. Reteste deve identificar a correção e executar regressão dos fluxos afetados. Sem execução, deixar as três opções de status desmarcadas; não confundir “não executado” com “bloqueado”.

8. Casos de teste

30 casos; 28 com evidência automatizada e 2 planejados. Status reflete somente o cenário e as asserções descritas. Executar a suíte completa para reproduzir sua preparação.

CT-001 - Cadastrar dados básicos

ID: CT-001

Título: Cadastrar dados básicos

Requisito(s): RF001

Pré-condição: API disponível e matrícula RF001-01 livre.

Dados de entrada: RF001-01; Ana Souza; Padrão; salarioBase=2500.

Passos:

1. Enviar POST /api/colaboradores com os dados.

2. Ler o status e o corpo JSON.

3. Conferir identificação, tipo, base e id.

Resultado esperado: HTTP 201; matrícula, nome, base e tipo preservados; id textual gerado.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF001 - cadastra colaborador com matrícula, nome, salário base e tipo.

CT-002 - Cadastrar colaborador Padrão

ID: CT-002

Título: Cadastrar colaborador Padrão

Requisito(s): RF002, RN008

Pré-condição: API disponível e matrícula RF002-01 livre.

Dados de entrada: RF002-01; Bruno Lima; Padrão; salarioBase=1800.

Passos:

1. Enviar POST /api/colaboradores.

2. Ler remuneração retornada.

3. Comparar adicional e final com a base.

Resultado esperado: HTTP 201; adicional=0; salarioFinal=1800.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF002 - cadastra colaborador padrão que recebe somente o salário base.

8. Casos de teste - continuação

CT-003 - Registrar campos de comissão

ID: CT-003

Título: Registrar campos de comissão

Requisito(s): RF003

Pré-condição: API disponível e matrícula RF003-01 livre.

Dados de entrada: RF003-01; Carla Mendes; Comissionado; base=2000; vendas=10000; percentual=5.

Passos:

1. Enviar POST com valorVendas e percentualComissao.

2. Ler o corpo retornado.

3. Conferir categoria e campos específicos.

Resultado esperado: HTTP 201; tipo Comissionado, vendas=10000 e percentual=5 preservados. A fórmula é verificada em CT-006.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF003 - cadastra colaborador comissionado com vendas e percentual.

CT-004 - Registrar campos de produção

ID: CT-004

Título: Registrar campos de produção

Requisito(s): RF004

Pré-condição: API disponível e matrícula RF004-01 livre.

Dados de entrada: RF004-01; Diego Alves; Produção; base=1500; quantidade=80; valorPorUnidade=12.5.

Passos:

1. Enviar POST com quantidadeProduzida e valorPorUnidade.

2. Ler o corpo retornado.

3. Conferir categoria e campos específicos.

Resultado esperado: HTTP 201; tipo Produção, quantidade=80 e valorPorUnidade=12.5 preservados. A fórmula é verificada em CT-006.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF004 - cadastra colaborador por produção com quantidade e valor unitário.

8. Casos de teste - continuação

CT-005 - Listar dados e remuneração

ID: CT-005

Título: Listar dados e remuneração

Requisito(s): RF005

Pré-condição: API isolada sem cadastros.

Dados de entrada: RF005-01; Elisa Ramos; Comissionado; base=2200; vendas=3000; percentual=10.

Passos:

1. Cadastrar o colaborador por POST.

2. Executar GET /api/colaboradores.

3. Conferir quantidade, identificação e remuneração.

Resultado esperado: HTTP 200; um registro com base=2200, adicional=300 e final=2500.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF005 - consulta a lista com dados cadastrais e remuneratórios.

CT-006 - Calcular as três remunerações

ID: CT-006

Título: Calcular as três remunerações

Requisito(s): RF006, RN008

Pré-condição: API isolada; três matrículas livres.

Dados de entrada: Base=1000 para todos; Padrão; Comissionado com vendas=2000 e percentual=10; Produção com quantidade=20 e valor=5.

Passos:

1. Cadastrar cada categoria com matrícula própria.

2. Ler o salário final de cada resposta.

3. Comparar com cálculo independente das três fórmulas.

Resultado esperado: Salários finais: Padrão=1000, Comissionado=1200 e Produção=1100.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF006 - calcula automaticamente o salário final conforme a categoria.

8. Casos de teste - continuação

CT-007 - Gerar folha detalhada consolidada

ID: CT-007

Título: Gerar folha detalhada consolidada

Requisito(s): RF007

Pré-condição: API isolada sem cadastros.

Dados de entrada: RF007-01/Fábio Castro/Padrão/base=2100; RF007-02/Gisele Moraes/Comissionado/base=2000/vendas=5000/percentual=4.

Passos:

1. Cadastrar os dois colaboradores.

2. Executar GET /api/folha-pagamento.

3. Comparar os itens e todos os campos remuneratórios.

Resultado esperado: HTTP 200; dois itens com matrícula, nome, tipo, base, adicional e final; finais 2100 e 2200.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF007 - gera folha detalhada para todos os colaboradores cadastrados.

CT-008 - Somar o total da folha

ID: CT-008

Título: Somar o total da folha

Requisito(s): RF008

Pré-condição: API isolada sem cadastros.

Dados de entrada: RF008-01/Helena Reis/Padrão/base=1800; RF008-02/Igor Nunes/Produção/base=1200/quantidade=100/valor=3.

Passos:

1. Cadastrar os dois colaboradores.

2. Executar GET /api/folha-pagamento.

3. Comparar quantidade e total do resumo.

Resultado esperado: HTTP 200; quantidadeColaboradores=2; totalFolha=3300.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF008 - emite resumo com o valor total da folha.

8. Casos de teste - continuação

CT-009 - Executar manutenção completa

ID: CT-009

Título: Executar manutenção completa

Requisito(s): RF009

Pré-condição: API isolada; RF009-01 livre.

Dados de entrada: Joana Freitas; Padrão; base inicial=1600; novo nome=Joana Freitas Atualizada; nova base=1900.

Passos:

1. Cadastrar RF009-01 e consultar sua matrícula.
2. Enviar PUT completo com novo nome e base; conferir retorno.
3. Enviar DELETE e consultar novamente.

Resultado esperado: Cadastro 201; consulta 200; alteração 200 e final=1900; exclusão 204; consulta final 404.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/requisitosFuncionais.test.ts :: RF009 - inclui, consulta, altera e exclui um cadastro.

CT-010 - Rejeitar matrícula duplicada

ID: CT-010

Título: Rejeitar matrícula duplicada

Requisito(s): RN001, RF001

Pré-condição: Existe FC-1007 na base de teste.

Dados de entrada: Nova matrícula fc-1007; Outro nome; Padrão; base=1000.

Passos:

1. Enviar POST com matrícula em minúsculas.
2. Ler status e mensagem.
3. Comparar com a rejeição de duplicidade esperada.

Resultado esperado: HTTP 409; mensagem A matrícula já está cadastrada.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: rejeita matrícula duplicada sem diferenciar maiúsculas.

8. Casos de teste - continuação

CT-011 - Rejeitar nome em branco

ID: CT-011

Título: Rejeitar nome em branco

Requisito(s): RN002, RNF002

Pré-condição: API disponível; RN-002 livre.

Dados de entrada: RN-002; nome com três espaços; Padrão; base=1000.

Passos:

1. Montar cadastro com nome somente espaços.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para nome inválido.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

CT-012 - Rejeitar salário base negativo

ID: CT-012

Título: Rejeitar salário base negativo

Requisito(s): RN003, RNF002

Pré-condição: API disponível; RN-003 livre.

Dados de entrada: RN-003; Nome válido; Padrão; salarioBase=-0.01.

Passos:

1. Montar cadastro com base negativa.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para salário base negativo.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

8. Casos de teste - continuação

CT-013 - Rejeitar vendas negativas

ID: CT-013

Título: Rejeitar vendas negativas

Requisito(s): RN004, RNF002

Pré-condição: API disponível; RN-004 livre.

Dados de entrada: RN-004; Nome válido; Comissionado; base=1000; vendas=-0.01; percentual=10.

Passos:

1. Montar cadastro com vendas negativas.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para valor de vendas negativo.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

CT-014 - Rejeitar percentual negativo

ID: CT-014

Título: Rejeitar percentual negativo

Requisito(s): RN005, RNF002

Pré-condição: API disponível; RN-005 livre.

Dados de entrada: RN-005; Nome válido; Comissionado; base=1000; vendas=100; percentual=-0.01.

Passos:

1. Montar cadastro com percentual negativo.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para percentual negativo.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

8. Casos de teste - continuação

CT-015 - Rejeitar quantidade negativa

ID: CT-015

Título: Rejeitar quantidade negativa

Requisito(s): RN006, RNF002

Pré-condição: API disponível; RN-006 livre.

Dados de entrada: RN-006; Nome válido; Produção; base=1000; quantidade=-1; valor=10.

Passos:

1. Montar cadastro com quantidade negativa.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para quantidade produzida negativa.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

CT-016 - Rejeitar valor unitário negativo

ID: CT-016

Título: Rejeitar valor unitário negativo

Requisito(s): RN007, RNF002

Pré-condição: API disponível; RN-007 livre.

Dados de entrada: RN-007; Nome válido; Produção; base=1000; quantidade=1; valor=-0.01.

Passos:

1. Montar cadastro com valor unitário negativo.
2. Enviar POST /api/colaboradores.
3. Conferir rejeição HTTP.

Resultado esperado: HTTP 400 para valor por unidade negativo.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: aplica RN002 a RN007 a todos os dados de remuneração. Cenário coberto dentro de teste agrupado; não é uma execução independente.

8. Casos de teste - continuação

CT-017 - Consultar folha individual atualizada

ID: CT-017

Título: Consultar folha individual atualizada

Requisito(s): RF005, RF006, RF007

Pré-condição: API isolada com PAD/Padrão/base=1000.25 e outros registros da suíte de relatórios.

Dados de entrada: GET /api/folha-pagamento/pad; PUT PAD com base=3000; DELETE PAD.

Passos:

1. Consultar folha de PAD e conferir seus seis campos.
2. Atualizar PAD; consultar folha e comparar final.
3. Excluir PAD e consultar folha novamente.

Resultado esperado: Detalhe 200; salário após alteração=3000; detalhe após exclusão=404.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/relatorios.test.ts :: relatórios individuais refletem cadastro, alteração, exclusão e total por categoria. Cenário coberto dentro de teste agrupado; não é uma execução independente.

CT-018 - Conferir resumo por categoria

ID: CT-018

Título: Conferir resumo por categoria

Requisito(s): RF008

Pré-condição: API isolada; cadastrar PAD, COM, PRO e PAD2.

Dados de entrada: Padrão: 1000.25 e 0.10; Comissionado: base=2000,vendas=1000,percentual=10; Produção: base=1500,quantidade=10,valor=2.5.

Passos:

1. Cadastrar os quatro registros.
2. Consultar a folha consolidada.
3. Comparar quantidade, total geral e totais por categoria.

Resultado esperado: Quantidade=4; total=4625.35; Padrão=1000.35, Comissionado=2100, Produção=1525.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/relatorios.test.ts :: relatórios individuais refletem cadastro, alteração, exclusão e total por categoria. Cenário coberto dentro de teste agrupado; não é uma execução independente.

8. Casos de teste - continuação

CT-019 - Gerar folha vazia

ID: CT-019

Título: Gerar folha vazia

Requisito(s): RF007, RF008

Pré-condição: JSON de teste contém [].

Dados de entrada: GET /api/folha-pagamento sem registros.

Passos:

1. Iniciar API com a base vazia.

2. Solicitar a folha.

3. Conferir itens, quantidade e totais.

Resultado esperado: HTTP 200; itens=[]; quantidade=0; totalFolha=0; três categorias com quantidade e total zero.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: gera folha vazia com totais zerados em todas as categorias.

CT-020 - Aceitar zero e persistir cadastro

ID: CT-020

Título: Aceitar zero e persistir cadastro

Requisito(s): RF002, RN003, RNF003

Pré-condição: API isolada; FC-1007 livre.

Dados de entrada: FC-1007; Mariana Oliveira; Padrão; salarioBase=0.

Passos:

1. Cadastrar por POST.

2. Conferir status, id, adicional e final.

3. Ler o JSON e comparar com o retorno.

Resultado esperado: HTTP 201; id gerado; adicional=0 e final=0; arquivo contém exatamente o cadastro retornado.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: cadastra e persiste um colaborador padrão.

8. Casos de teste - continuação

CT-021 - Alterar categoria preservando identidade

ID: CT-021

Título: Alterar categoria preservando identidade

Requisito(s): RF009, RF006

Pré-condição: FC-1007 existe; guardar id/criadoEm; base com três colaboradores.

Dados de entrada: Renomear para FC-2007; Mariana Oliveira Atualizada; Comissionado; base=1000; vendas=2000; percentual=10.

Passos:

1. Enviar PUT /api/colaboradores/fc-1007 com os dados completos.

2. Comparar id, criadoEm, atualizadoEm e remuneração.

3. Consultar matrículas antiga/nova e conferir JSON.

Resultado esperado: HTTP 200; identidade preservada; atualizadoEm definido; adicional=200, final=1200; antiga 404/nova 200; três registros persistidos.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: altera um colaborador, preserva sua identidade e recalcula o salário.

CT-022 - Impedir duplicidade na alteração

ID: CT-022

Título: Impedir duplicidade na alteração

Requisito(s): RN001, RF009

Pré-condição: FC-2007 e FC-1008 existem.

Dados de entrada: PUT FC-2007 com matrícula fc-1008; Mariana Oliveira Atualizada; Padrão; base=1000.

Passos:

1. Preparar alteração para matrícula de outro registro.

2. Enviar PUT completo.

3. Ler status e mensagem.

Resultado esperado: HTTP 409; mensagem A matrícula já está cadastrada.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: impede matrícula duplicada durante uma alteração.

8. Casos de teste - continuação

CT-023 - Tratar manutenção de registro inexistente

ID: CT-023

Título: Tratar manutenção de registro inexistente

Requisito(s): RF009

Pré-condição: NAO-EXISTE não está na base.

Dados de entrada: PUT NAO-EXISTE com FC-9999/Nome válido/Padrão/base=1000; DELETE NAO-EXISTE.

Passos:

1. Enviar alteração completa para matrícula inexistente.
2. Enviar exclusão para a mesma matrícula.
3. Conferir ambos os status.

Resultado esperado: PUT e DELETE retornam 404.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: responde 404 ao alterar ou excluir cadastro inexistente.

CT-024 - Recuperar dados em nova instância

ID: CT-024

Título: Recuperar dados em nova instância

Requisito(s): RNF003, RF005

Pré-condição: Arquivo temporário compartilhado apenas entre as duas aplicações do teste.

Dados de entrada: ALIAS; Nome; tipoColaborador=Padrão; salarioBase=100.

Passos:

1. Cadastrar na primeira aplicação.
2. Criar segunda aplicação apontando ao mesmo JSON.
3. Consultar ALIAS pela segunda aplicação; encerrar e limpar.

Resultado esperado: Cadastro 201; consulta pela nova aplicação 200; salarioFinal=100. Testa reabertura do arquivo, não reinício completo do sistema operacional.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: mantém compatibilidade com tipoColaborador e com dados após reiniciar.

8. Casos de teste - continuação

CT-025 - Rejeitar tipos inválidos sem gravar

ID: CT-025

Título: Rejeitar tipos inválidos sem gravar

Requisito(s): RNF002, RN004

Pré-condição: API com arquivo JSON válido; guardar seu conteúdo antes.

Dados de entrada: Comissionado; INVALIDO/Teste/base=1000/percentual=10; valorVendas em null,true,[],{},espaço,abc,-1.

Passos:

1. Enviar um POST para cada valor inválido.
2. Verificar 400, mensagem e erro de campo valorVendas.
3. Comparar arquivo final com o conteúdo inicial.

Resultado esperado: Todos retornam 400 com erro de valorVendas; JSON permanece idêntico.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: express-validator retorna erros de campos sem alterar o arquivo.

CT-026 - Preservar arquivo corrompido ao falhar

ID: CT-026

Título: Preservar arquivo corrompido ao falhar

Requisito(s): RNF003

Pré-condição: JSON temporário válido; guardar conteúdo para restaurar.

Dados de entrada: Substituir temporariamente por {invalido; GET /api/colaboradores.

Passos:

1. Corromper somente o arquivo temporário do teste.
2. Solicitar listagem e conferir status e conteúdo do arquivo.
3. Restaurar conteúdo original no bloco de limpeza.

Resultado esperado: HTTP 500; arquivo permanece {invalido até a restauração pelo teste.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: arquivo corrompido resulta em 500 e não é sobrescrito.

8. Casos de teste - continuação

CT-027 - Normalizar entrada e impedir salário forjado

ID: CT-027

Título: Normalizar entrada e impedir salário forjado

Requisito(s): RN008, RNF002, RF001

Pré-condição: Matrícula VALIDATOR livre.

Dados de entrada: matricula e nome com espaços; Padrão; salarioBase=string 1234.50; id=forjado; salarioFinal=1; valorVendas=999; campoExtra=ignorar.

Passos:

1. Enviar POST com campos legítimos e extras.

2. Ler identificação, base e salário calculado.

3. Conferir remoção dos campos indevidos; excluir dado de teste.

Resultado esperado: HTTP 201; textos aparados; base/final=1234.5; id diferente de forjado; campoExtra e valorVendas ausentes.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: express-validator normaliza campos e descarta dados não autorizados.

CT-028 - Recalcular valores derivados ao consultar

ID: CT-028

Título: Recalcular valores derivados ao consultar

Requisito(s): RF006, RN008

Pré-condição: Comissionado base=2000/vendas=10500/percentual=7.5 e Produção base=1800/quantidade=180/valor=12.35 persistidos.

Dados de entrada: Alterar adicional e salarioFinal dos dois registros no JSON temporário para 1.

Passos:

1. Modificar somente os campos derivados no arquivo de teste.

2. Solicitar GET /api/colaboradores.

3. Conferir os valores recalculados das duas categorias.

Resultado esperado: HTTP 200; Comissionado adicional=787.5/final=2787.5; Produção adicional=2223/final=4023.

Resultado obtido: Asserções do cenário aprovadas na rodada de 29/09/2026 19:35:14 (America/Sao_Paulo).

Status: (X) Passou ( ) Falhou ( ) Bloqueado

Evidência: backend/test/colaboradoresApi.test.ts :: recalcula salários ao consultar os registros persistidos.

8. Casos de teste - continuação

CT-029 - Validar usabilidade dos fluxos principais

ID: CT-029

Título: Validar usabilidade dos fluxos principais

Requisito(s): RNF001

Pré-condição: Frontend e API locais; operador disponível; dados fictícios; telas 1366×768 e 390×844.

Dados de entrada: UX-01; Pessoa Teste; Padrão; base=1500; repetir envio com nome vazio.

Passos:

1. Operador localiza cadastro, preenche campos rotulados e salva; registrar dificuldade.
2. Consultar o registro e abrir folha individual/consolidada em ambas as larguras.
3. Enviar nome vazio, conferir mensagem compreensível, legibilidade e ausência de sobreposição.

Resultado esperado: Operador conclui cadastro, consulta e folha; campos e ações identificáveis; erro explica correção; conteúdo legível nos dois tamanhos. Registrar observações e decisão do operador.

Resultado obtido: Não executado. Preencher após execução planejada, com observações e evidência.

Status: ( ) Passou ( ) Falhou ( ) Bloqueado

Evidência: Execução manual/revisão planejada.

CT-030 - Revisar extensão das categorias

ID: CT-030

Título: Revisar extensão das categorias

Requisito(s): RNF004

Pré-condição: Código disponível; revisão por desenvolvimento e QA; ramo de experimento isolado.

Dados de entrada: Categoria experimental com adicional fixo=100; base=1000; exemplos atuais do CT-006.

Passos:

1. Identificar subclasse, fábrica, tipos, validações e interface como pontos de extensão.
2. No ramo isolado, introduzir categoria experimental sem editar as fórmulas das três classes existentes.
3. Conferir diff, testar final=1100 da nova categoria e reexecutar regressão existente; registrar revisão.

Resultado esperado: Nova categoria calcula 1100; fórmulas existentes inalteradas e testes antigos aprovados. Documentar pontos alterados e parecer do revisor; experimento não entra na baseline automaticamente.

Resultado obtido: Não executado. Preencher após execução planejada, com observações e evidência.

Status: ( ) Passou ( ) Falhou ( ) Bloqueado

Evidência: Execução manual/revisão planejada.

9. Planejamento do projeto

Base do planejamento

Data de planejamento: 29/09/2026. Prazo A1: 20/10/2026. O código e a suíte já existem; o cronograma abaixo reserva consolidação, revisão e entrega. As datas futuras são previstas, não registros fictícios de trabalho concluído.

EAP - Estrutura Analítica do Projeto

1. Gestão da A1: 1.1 identificar equipe; 1.2 atribuir papéis; 1.3 acompanhar riscos; 1.4 conferir submissão.
2. Requisitos: 2.1 visão; 2.2 ERS; 2.3 critérios de aceitação; 2.4 rastreabilidade.
3. Análise e projeto: 3.1 casos de uso; 3.2 classes; 3.3 sequência; 3.4 arquitetura/persistência.
4. Qualidade: 4.1 PGQ; 4.2 plano de testes; 4.3 especificar 30 CTs; 4.4 coletar evidências.
5. Repositório: 5.1 README; 5.2 estrutura; 5.3 conferir histórico; 5.4 empacotar fontes.
6. Fechamento: 6.1 revisão cruzada; 6.2 ajustes; 6.3 conferência final; 6.4 entrega.

Cronograma e dependências

EAP	Atividade	Início	Fim	Dependência	Responsável	Saída
1	Identificação e atribuição nominal	29/09	01/10	Nenhuma	Coordenação	Equipe e papéis preenchidos
2	Revisar visão, ERS e critérios	29/09	03/10	Fonte do cliente	Análise	Baseline de requisitos revisada
3	Conferir UML e arquitetura	02/10	06/10	2	Desenvolvimento	Modelos coerentes com código
4.1-4.3	Revisar PGQ, plano e CTs	04/10	08/10	2 e 3	QA	Critérios e 30 casos conferidos
4.4	Reexecutar automação se houver mudanças	09/10	12/10	4.1-4.3	QA + Desenvolvimento	Evidências atualizadas
5	Conferir README, acesso e pacote	10/10	14/10	3 e 4	Configuração	Repositório reproduzível
6.1-6.2	Revisão cruzada e ajustes	15/10	17/10	2-5	Revisor	Pendências resolvidas
6.3	Reserva e conferência final	18/10	19/10	6.1-6.2	Coordenação	Checklist e identificação completos
6.4	Submissão A1	20/10	20/10	6.3	Coordenação	Comprovante de entrega

Todas as datas da tabela são de 2026. Reunião curta de acompanhamento semanal, com revisão de pendências e registro de decisões. Desvio que comprometa 17/10 é comunicado à coordenação; 18-19/10 são reserva de ajuste, não escopo novo.

Responsabilidades

Papel	Executa	Revisa / aprova
Coordenação	Cronograma, atribuição nominal e envio	Checklist final e baseline
Análise	Visão, ERS e rastreabilidade	Aderência dos CTs ao cliente
Desenvolvimento	UML, arquitetura e correções	Viabilidade técnica e fórmulas
QA	PGQ, plano, CTs, execução e evidências	Critérios de entrada/saída
Configuração	README, dependências, histórico e ZIP	Reprodutibilidade e conteúdo do pacote
Revisor	Revisão cruzada independente do autor quando possível	Consistência e registro de não conformidades

Distribuição proposta para execução do plano: Felipe Vergara Chico assume Coordenação e QA; Nicolas Uliana Ramos assume Análise e documentação; Victor Pithan Altnetter assume Desenvolvimento frontend e revisão da interface; Miguel Marques Durte assume Desenvolvimento backend e Configuração. Nicolas revisa os CTs de Felipe; Felipe revisa a ERS de Nicolas; Victor e Miguel revisam mutuamente arquitetura e contratos. Essa distribuição é planejamento, não registro de autoria passada ou aprovação já realizada.

Riscos e resposta

Risco	Probabilidade / impacto	Prevenção / resposta	Dono
Ambiguidade de percentual/precisão	Média / alto	Confirmar D01-D06 e manter exemplos de cálculo	Análise
Alterações invalidarem evidências	Média / alto	Rodar testes e regenerar relatório após mudanças	QA
Dados JSON perdidos por múltiplos processos	Baixa no uso local / alto	Uma API por arquivo; backup antes de demonstrações	Desenvolvimento
Dependências ou portas indisponíveis	Média / médio	Instalar antes da aula e verificar HTTP local	Configuração
Identificação/acesso faltando	Média / alto	Conferência nominal e de acesso até 14/10	Coordenação
Escopo A3 invadir fechamento A1	Média / médio	Registrar backlog separado e congelar documentos em 17/10	Coordenação

A A3 (01/12/2026) requer implementação final, testes de aceitação, defeitos, manuais e homologação. Estes itens ficam para planejamento posterior, sem comprometer o fechamento dos nove entregáveis da A1.

Verificações da preparação A1

Data: 29/09/2026. Código-base: commit 1257880 (ver hash completo no histórico). Nesta preparação foram adicionados documentos e gerador de entrega, sem alterar a implementação de negócio ou os testes existentes.

Projeto	Comando	Resultado observado
backend	npm run typecheck	Código de saída 0
backend	npm run lint	Código de saída 0
backend	npm run build	Código de saída 0
frontend	npm run typecheck	Código de saída 0
frontend	npm run lint	Código de saída 0
frontend	npm run build	Código de saída 0; 56 módulos transformados
backend	npm run test:coverage com reporter JSON	52 aprovados; 0 falhas; 0 ignorados; 4 arquivos

Comando exato da execução registrada:

npm run test:coverage -- --coverage.reportsDirectory=/tmp/folha-clara-a1-coverage --reporter=default --reporter=json --outputFile=/tmp/folha-clara-a1-vitest.json

Foi necessário executar com permissão de escuta HTTP local. Uma tentativa restrita foi interrompida por indisponibilidade das portas; a rodada válida usou pasta de cobertura isolada para não conflitar com o bloqueio anterior. O JSON anexado é da rodada válida, não da tentativa interrompida.

Cobertura global observada: instruções 98,53% (269/273); ramificações 96,99% (129/133); funções 98,64% (73/74); linhas 98,82% (253/256). Os limites de 80% por arquivo foram aprovados. Nenhuma cobertura de frontend foi medida.

vitest-resultados.json preserva asserções e status; **coverage-final.json** preserva contadores por arquivo; **cobertura-resumo.json** é uma agregação legível. CT-011 a CT-016 compartilham um teste que executa seis entradas inválidas; CT-017/CT-018 compartilham o teste de relatórios. Portanto, 28 CTs com evidência automatizada não equivalem a 28 testes Vitest independentes nem a toda a suíte de 52 testes. CT-029/CT-030 não foram executados.

10. Evidências e proveniência

Rodada Vitest iniciada em 29/09/2026 19:35:14 (America/Sao_Paulo): 52 aprovados de 52; 0 falhas e 0 pendentes. Detalhes por asserção em 04_Testes/evidencias/vitest-resultados.json no ZIP.

Métrica backend	Cobertos / total	Percentual
Instrucoes	269 / 273	98.53%
Ramificacoes	129 / 133	96.99%
Funcoes	73 / 74	98.64%
Linhas	253 / 256	98.82%

Histórico recente real do repositório

1257880 · 2026-09-29 · Relatorio de testes

94dd328 · 2026-09-22 · feat: add Vitest configuration and coverage support, implement coverage tests for API

cd3fda1 · 2026-09-22 · feat: implement functional requirements tests for payroll system

1c76ad3 · 2026-09-22 · refactor: migrate tests to use Vitest framework

5e2ed40 · 2026-09-22 · Merge pull request #12 from miguel-mduarte/feature/completar-relatorios-folha

2a243cb · 2026-09-22 · feat: add individual payroll generation and reporting features

cbaa639 · 2026-09-09 · Merge pull request #11 from miguel-mduarte/fix/javascript-to-typescript

9699623 · 2026-09-09 · Transformando codigo Javascript para Typescript

O histórico completo acompanha o pacote. Os novos artefatos desta A1 são arquivos de trabalho preparados sobre a revisão indicada, sem criação de commits ou submissão remota automática. O manifesto SHA-256 identifica os arquivos entregues, inclusive documentação ainda não commitada.

Referências documentais

1. Trabalho_Integrador_A1_A3_GQS.pdf, documento de 28/09/2026: seção 5 (A1), seção 7 (estrutura) e seção 8 (avaliação). 2. T1 - Requisitos do Cliente.docx: objetivo, escopo, requisitos, regras, relatórios e aceitação. 3. Código local backend/src, frontend/src e backend/test; arquivos de dependências e histórico Git anexados.